

EcoNest ML Scripts

Real-time smart home analytics and alerting system. Monitors sensor data, detects anomalies, runs AI-powered inference via a local LLM (Mistral/Ollama), and sends SMS alerts.

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Prerequisites](#)
- 3. [Installation](#)
- 4. [Configuration](#)
- 5. [Database Schema](#)
- 6. [Scripts](#)
 - [conn_check.py](#)
 - [analytics.py](#)
 - [trigger.py](#)
 - [inference.py](#)
 - [demo.py](#)
- 7. [Running the System](#)
- 8. [SMS Setup](#)
- 9. [Deployment Notes](#)
- 10. [Troubleshooting](#)

Architecture Overview



Two-tier design: `trigger.py` is the fast detector (lightweight, no LLM). `inference.py` is the smart classifier (full context, Mistral, SMS gating).

Prerequisites

Requirement	Version	Notes
-------------	---------	-------

Requirement	Version	Notes
MySQL	8.0+	Schema described below
Home Assistant	Any recent	Running with long-lived access token
Ollama	Any	With mistral model pulled
Gmail account	—	App password enabled (for SMS)

Installation

```
cd machine_learning/scripts

pip install -r requirements.txt
```

Pull the Mistral model into Ollama (one-time):

```
ollama pull mistral
```

Configuration

Create a `.env` file in the same directory as the scripts (or in `~/` `.env`):

```
# MySQL
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=econest
DB_NAME=econest
DB_PORT=3306

# Home Assistant
HA_URL=http://localhost:8123
HA_TOKEN=your_long_lived_access_token_here

# Gmail SMTP (for SMS alerts)
SMS_GMAIL=your-gmail@gmail.com
SMS_GMAIL_APP_PASSWORD=xxxx-xxxx-xxxx-xxxx

# SMS destination (carrier email gateway – see SMS Setup below)
SMS_TO=5551234567@tmomail.net
```

Gmail app password: In your Google account go to Security → 2-Step Verification → App passwords. Generate one for "Mail".

Carrier SMS gateways (replace `NUMBER` with 10-digit phone number):

Carrier	Gateway
T-Mobile	NUMBER@tmomail.net
Verizon	NUMBER@vtext.com
AT&T	NUMBER@txt.att.net
Cricket	NUMBER@sms.cricketwireless.net
Boost	NUMBER@smsmyboostmobile.com
Metro PCS	NUMBER@mymetropcs.com

Carrier	Gateway
US Cellular	NUMBER@email.uscc.net
Virgin Mobile	NUMBER@vmobl.com

Database Schema

The scripts read from and write to the following MySQL tables. Your backend is responsible for creating and populating sensor data; the ML scripts only write to `home_snapshot` and `home_analytics`.

Tables read by the scripts

sensor_readings — Raw sensor data from devices

room_id	INT	
device_id	INT	
data	JSON	-- e.g. {"power": 120.5, "motion": 1, "sound_level": 0.3}
created_at	DATETIME	

devices — Device registry

id	INT	
room_id	INT	
type	VARCHAR	-- "energy", "motion", "sound", etc.
is_active	TINYINT	

rooms — Room registry

id	INT	
name	VARCHAR	
is_active	TINYINT	

device_profiles — Device schedules

device_id	INT	
room_id	INT	
schedule	JSON	-- e.g. [{"start": "08:00", "end": "22:00"}]

home_snapshot — Written by trigger.py, read by inference.py

room_id	INT	
active_devices	JSON	
power_trend	FLOAT	
anomaly_detected	TINYINT	
anomaly_reason	VARCHAR	
updated_at	DATETIME	

home_analytics — Written by analytics.py, read by inference.py and trigger.py

room_id	INT	
hour	INT	-- 0-23
avg_power	FLOAT	
weekly_kwh	FLOAT	
hourly_pattern	JSON	-- array of 24 avg-power values
updated_at	DATETIME	

Scripts

conn_check.py

Quick sanity check — confirms your `.env` credentials can reach MySQL.

```
python conn_check.py
```

Output:

```
Connection successful!
Tables: ['devices', 'home_analytics', 'home_snapshot', ...]
```

Run this first whenever setting up a new environment or debugging connection errors.

analytics.py

Computes 7-day historical baselines per room per hour and stores them in `home_analytics`. Designed to run on a cron schedule (hourly or daily).

```
python analytics.py
```

What it computes per room:

- Average power for the current hour across the last 7 days
- Total kWh consumed over the last 7 days
- Hourly power pattern (24-element array, index = hour of day)
- Kitchen (room 1): also averages hourly sound levels
- Front Door (room 7): also averages hourly motion event ratios

Recommended cron schedule:

```
0 * * * * /usr/bin/python3 /path/to/scripts/analytics.py
```

trigger.py

Real-time anomaly detector. Runs as a daemon, polling sensor data every 30 seconds. When an anomaly is found, it writes to `home_snapshot` and spawns `inference.py --mode alert`.

```
python trigger.py
```

Anomaly thresholds:

- Power: `current_power > baseline * 4 AND current_power > 100W AND baseline > 20W`
- Security: motion + sound spike (1.5× sound baseline) at any hour
- Security: motion alone during night hours (23:00–06:00)

Config flags at top of file:

```
DRY_RUN = False # Set True to log anomalies without writing to DB or spawning inference
```

Note: The subprocess spawn path is hardcoded to macOS Homebrew (`/opt/homebrew/bin/python3`). For Linux/Windows deployment, update the `subprocess.Popen` call in `run()` to use the correct Python interpreter path.

inference.py

Production inference daemon. Runs as a continuous loop (30-second intervals), auto-detects whether to run in alert or routine mode, calls Mistral via Ollama, and sends SMS for HIGH severity events.

```
# Auto-detect mode (recommended)
python inference.py

# Force alert mode
python inference.py --mode alert

# Force routine recommendation mode
python inference.py --mode routine
```

Modes:

Mode	Trigger	Behavior
alert	Anomalies in home_snapshot	Classify anomaly type (SECURITY/FAULT/ENERGY), determine severity, SMS if HIGH
routine	No anomalies	Review all devices and baselines, surface one efficiency recommendation

Scheduled checks (time-gated, run each loop iteration):

Check	When	What it does
Laundry done	Any time	Detects washer/dryer finishing, sends pricing-aware SMS
Laundry peak alert	Any time	Alerts if laundry running during expensive hours
Sprinkler	7:00 AM only	Reads soil moisture + weather forecast → water/skip/delay decision
Wind-down	23:00–02:00	Dims lights, turns off TV, adjusts thermostat
Security	Any time	Cross-checks garage + motion + occupancy + power draw

SMS severity gates (code-level, not model-level):

- 1. Only HIGH severity events send SMS
- 2. Severity is upgraded to HIGH if an off-schedule device is running at night
- 3. If the model outputs SMS: YES but severity is not HIGH, SMS is suppressed (hallucination guard)

Electricity pricing tiers (Texas TOU):

Off-peak:	21:00–06:00 → 8¢/kWh
Mid-peak:	06:00–14:00, 20:00–21:00 → 12¢/kWh
Peak:	14:00–20:00 → 18¢/kWh

Config flags at top of file:

PAUSED = True # Set False to enable SMS sending in production

Log file: ~/inference.log — all decisions, SMS attempts, and errors.

demo.py

Interactive showcase of all six scenarios using live home data. Intended for demonstrations, not production use.

Basic (no SMS) python demo.py
With SMS output python demo.py --phone 5551234567 --carrier tmobile
With explicit gateway python demo.py --gateway 5551234567@tmomail.net

CLI arguments:

Argument	Description
--phone NUMBER	10-digit phone number
--carrier CARRIER	Carrier name: tmobile, verizon, cricket, boost, metro, sprint, uscellular, virgin
--gateway EMAIL	Full SMS gateway address (overrides --phone / --carrier)

Six scenes:

Scene	Description	SMS?
1	Routine efficiency recommendation	No
2	Energy anomaly alert + 2am simulation	If HIGH
3	Laundry running at peak pricing	No
4	Sprinkler morning check (7am)	If watering
5	Late-night wind-down (dry run)	No
6	Security check at 2am (dry run)	If HIGH

The demo pauses between scenes and prompts you to continue. Each scene prints a live home-state dashboard followed by Mistral's reasoning and recommended actions.

Running the System

Recommended production setup (three processes):

```
# Terminal 1 – anomaly detector (always running)
python trigger.py

# Terminal 2 – inference daemon (always running)
python inference.py

# Cron – analytics refresh (hourly)
0 * * * * /usr/bin/python3 /path/to/scripts/analytics.py
```

Before starting, verify everything is reachable:

```
python conn_check.py           # MySQL
curl http://localhost:8123/api/ # Home Assistant
curl http://localhost:11434/api/tags # Ollama
```

Enable SMS in inference.py when you're ready for production:

```
# inference.py, line ~40
PAUSED = False
```

SMS Setup

SMS is delivered via Gmail SMTP to carrier email-to-SMS gateways. No third-party SMS API is required.

1. Enable 2-Step Verification on your Gmail account.
2. Generate an App Password (Google Account → Security → App passwords).
3. Set SMS_GMAIL and SMS_GMAIL_APP_PASSWORD in your .env .
4. Set SMS_TO to PHONENUMBER@CARRIER_GATEWAY (see carrier table above).
5. Set PAUSED = False in inference.py .

Test it manually:

```
from inference import send_sms
send_sms("Test message from EcoNest")
```

Deployment Notes

Python interpreter path in trigger.py

`trigger.py` spawns `inference.py` via a hardcoded macOS path:

```
subprocess.Popen(["/opt/homebrew/bin/python3", "/Users/econest/scripts/inference.py", "--mode", "alert"])
```

Update this to match your environment before deploying:

```
# Linux example
subprocess.Popen(["/usr/bin/python3", "/home/econest/scripts/inference.py", "--mode", "alert"])

# Windows example
subprocess.Popen(["python", "C:/path/to/scripts/inference.py", "--mode", "alert"])
```

Home Assistant entity IDs

`inference.py` and `demo.py` reference specific entity IDs (e.g., `climate.ecobee`, `light.hue_color_spot_1`, `valve.side_r_lawn`). These must match your actual Home Assistant setup. Search for `entity_id` references in both files and update as needed.

Running as a service (Linux systemd)

Create `/etc/systemd/system/econest-trigger.service`:

```
[Unit]
Description=EcoNest Trigger
After=network.target mysql.service

[Service]
ExecStart=/usr/bin/python3 /home/econest/scripts/trigger.py
Restart=always
User=econest
EnvironmentFile=/home/econest/.env

[Install]
WantedBy=multi-user.target
```

Repeat for `inference.py`. Then:

```
systemctl enable econest-trigger econest-inference
systemctl start econest-trigger econest-inference
```

Troubleshooting

Connection failed from `conn_check.py`

- Confirm MySQL is running and the credentials in `.env` are correct.
- Check `DB_PORT` (default 3306).

`inference.py` never sends SMS

- Check `PAUSED = True` in `inference.py` — set it to `False`.
- Confirm `SMS_GMAIL_APP_PASSWORD` is an App Password, not your Gmail login password.
- Check `~/inference.log` for SMTP errors.

Ollama / Mistral errors

- Verify Ollama is running: `curl http://localhost:11434/api/tags`
- Verify the model is downloaded: `ollama list` should show `mistral`.
- Re-pull if needed: `ollama pull mistral`

Anomalies never trigger

- Run `analytics.py` first to populate `home_analytics` — without baselines, no anomalies can be computed.
- Confirm `home_snapshot` rows have `anomaly_detected = 1` (check DB directly).
- Check that `DRY_RUN = False` in `trigger.py`.

`trigger.py` spawns `inference` but nothing happens

- The subprocess path may be wrong for your OS (see Deployment Notes above).
- Check `~/inference.log` for errors from the spawned process.

Home Assistant entity not found

- Use `curl -H "Authorization: Bearer YOUR_TOKEN" http://localhost:8123/api/states` to list all entity IDs.
- Update hardcoded entity references in `inference.py` and `demo.py` to match.